EXPERIMENT NO. 07

Monitoring using Nagios with Docker and WSL2

LO Mapping: LO5

Name:Khan Ashhad
Roll NO:2401048
Subject:ASD Lab
Branch:AI&DS

Aim

To implement monitoring using Prometheus / Grafana / Nagios.

LO Mapping

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory

Continuous monitoring is a process to detect, report, and respond to events and problems that occur in infrastructure. Once an application is deployed on a server, continuous monitoring helps maintain the infrastructure and respond appropriately.

Why We Need Nagios Tool?

- Detects network or server issues.
- Helps identify the root cause of problems and supports permanent solutions.
- Provides active monitoring of infrastructure and business processes.
- Helps monitor and troubleshoot server performance issues.
- Supports planning for infrastructure upgrades before outdated systems create failures.
- Helps maintain service security and availability.
- Nagios is an open-source monitoring system capable of monitoring applications, services, and IT infrastructure.
- NRPE (Nagios Remote Plugin Executor) is designed to execute plugins on remote systems. On Windows hosts, NSClient++ can act as the remote monitoring agent.

Note on Running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. In this experiment, Nagios Core was run inside a Docker container using Docker Desktop with the WSL2 backend.

1. Pre-requisites: WSL2 and Docker Desktop

WSL2 was enabled and verified before installing Docker Desktop. Virtualization was enabled on the system, and Docker Desktop was configured to use the WSL2-based engine.

```
wsl --status
wsl --version
```

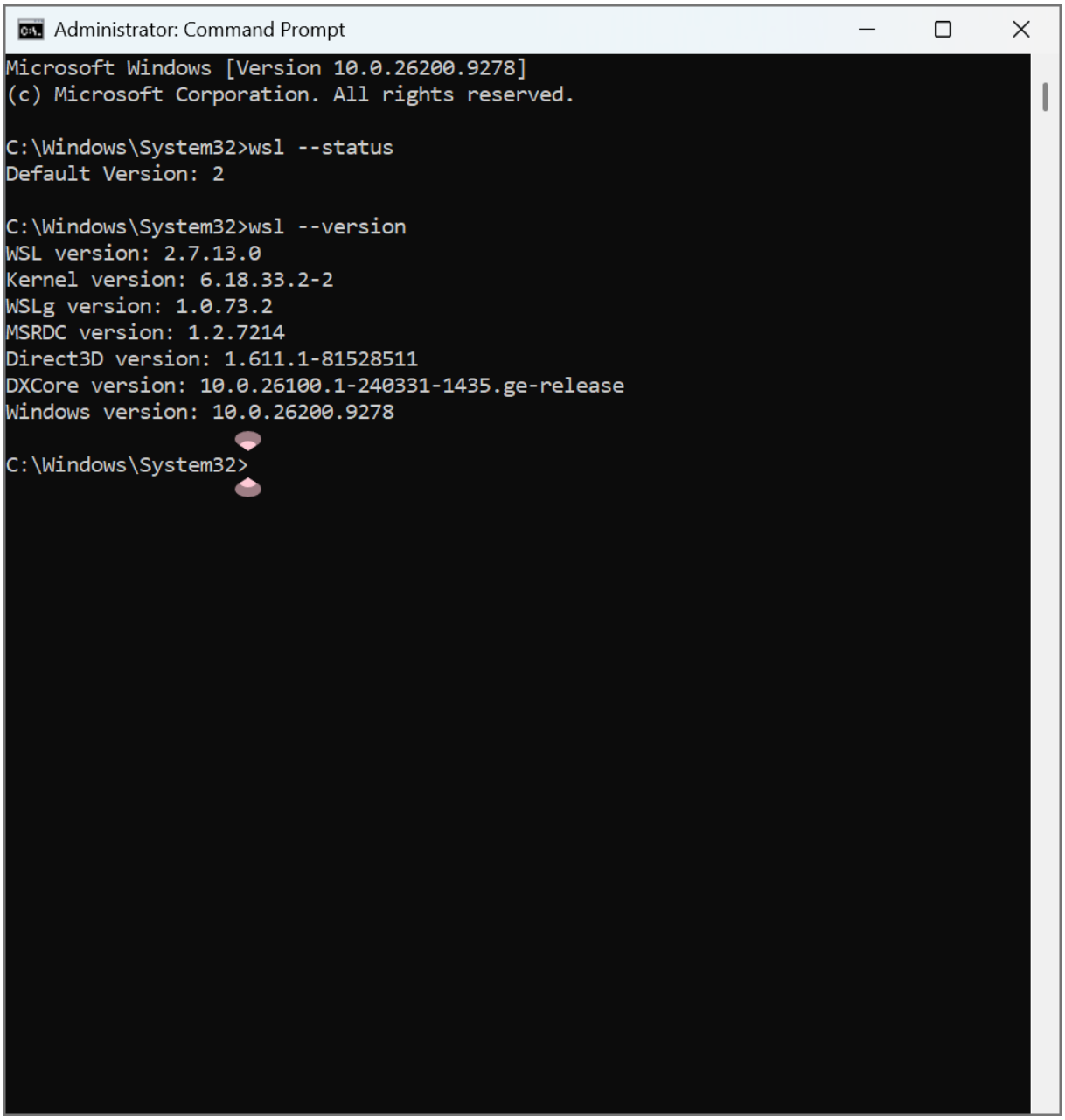


Fig. 1: WSL2 status and version verification

Docker Desktop was then installed and the Docker Engine was started successfully.

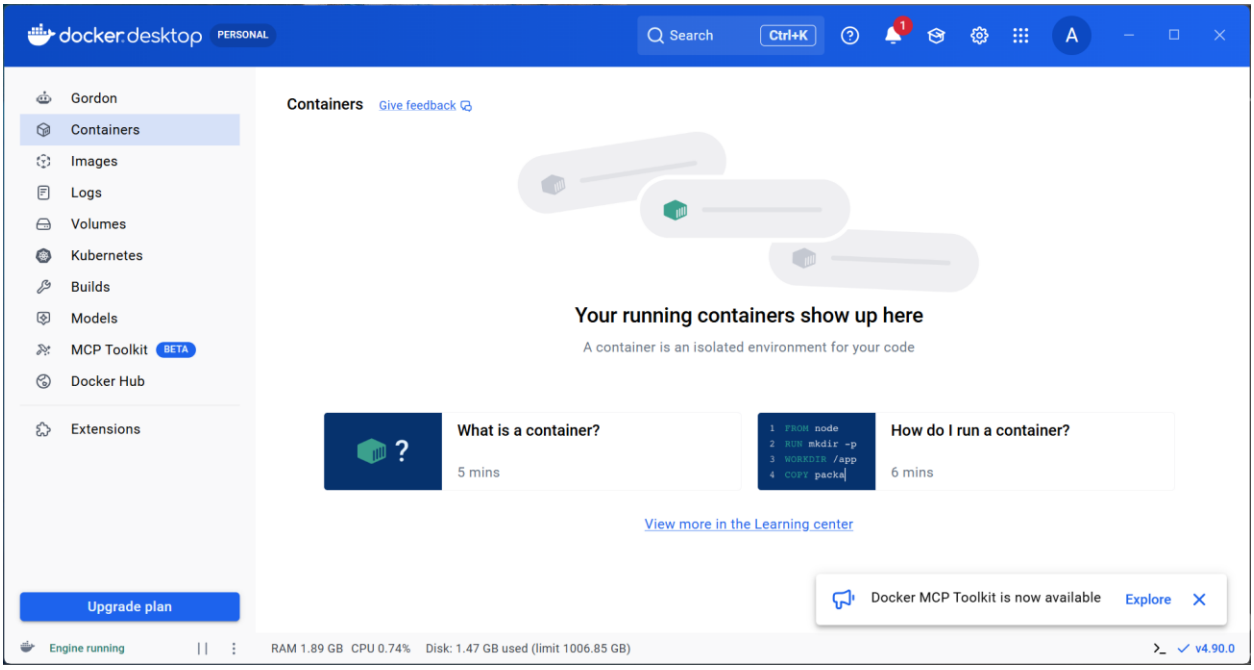


Fig. 2: Docker Desktop with Engine running

2. Pull and Run the Nagios Core Docker Image

A ready-made Nagios Core Docker image was downloaded and used to create a Nagios container.

```
docker pull jasonrivers/nagios:latest
```

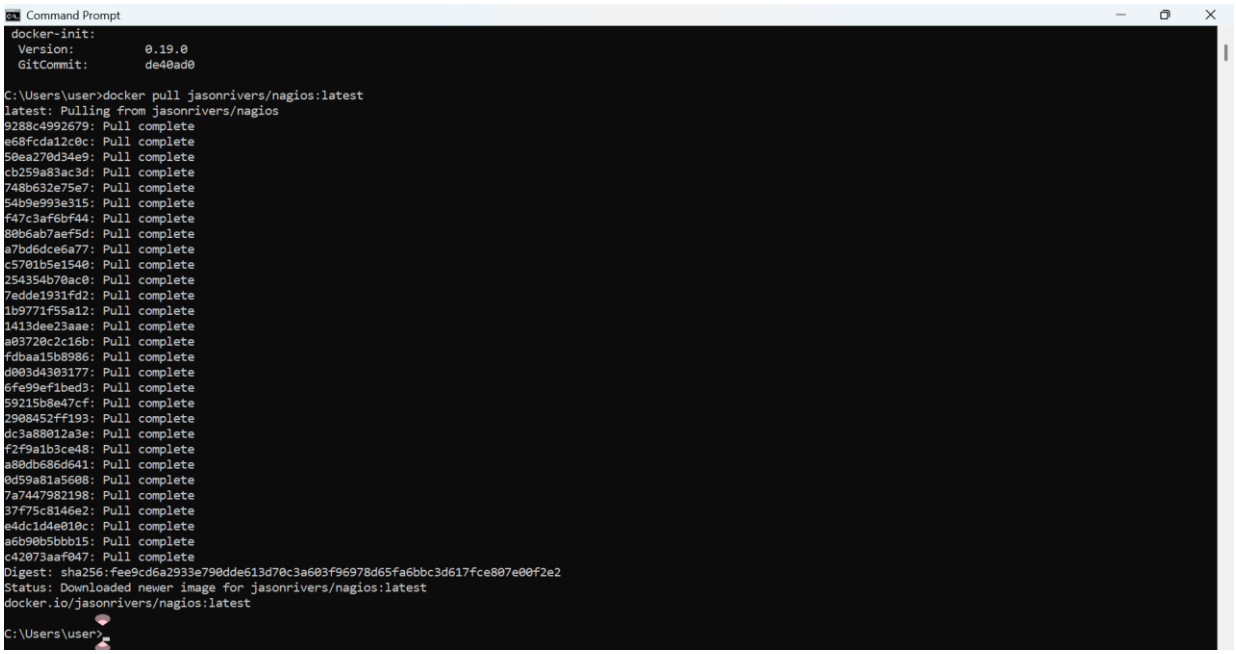


Fig. 3: Nagios Docker image pulled successfully

```
docker run -d --name nagios -p 8080:80 jasonrivers/nagios:latest
docker ps
```

The container was verified using docker ps. The Nagios container was running and port 8080 of the Windows host was mapped to port 80 of the container.

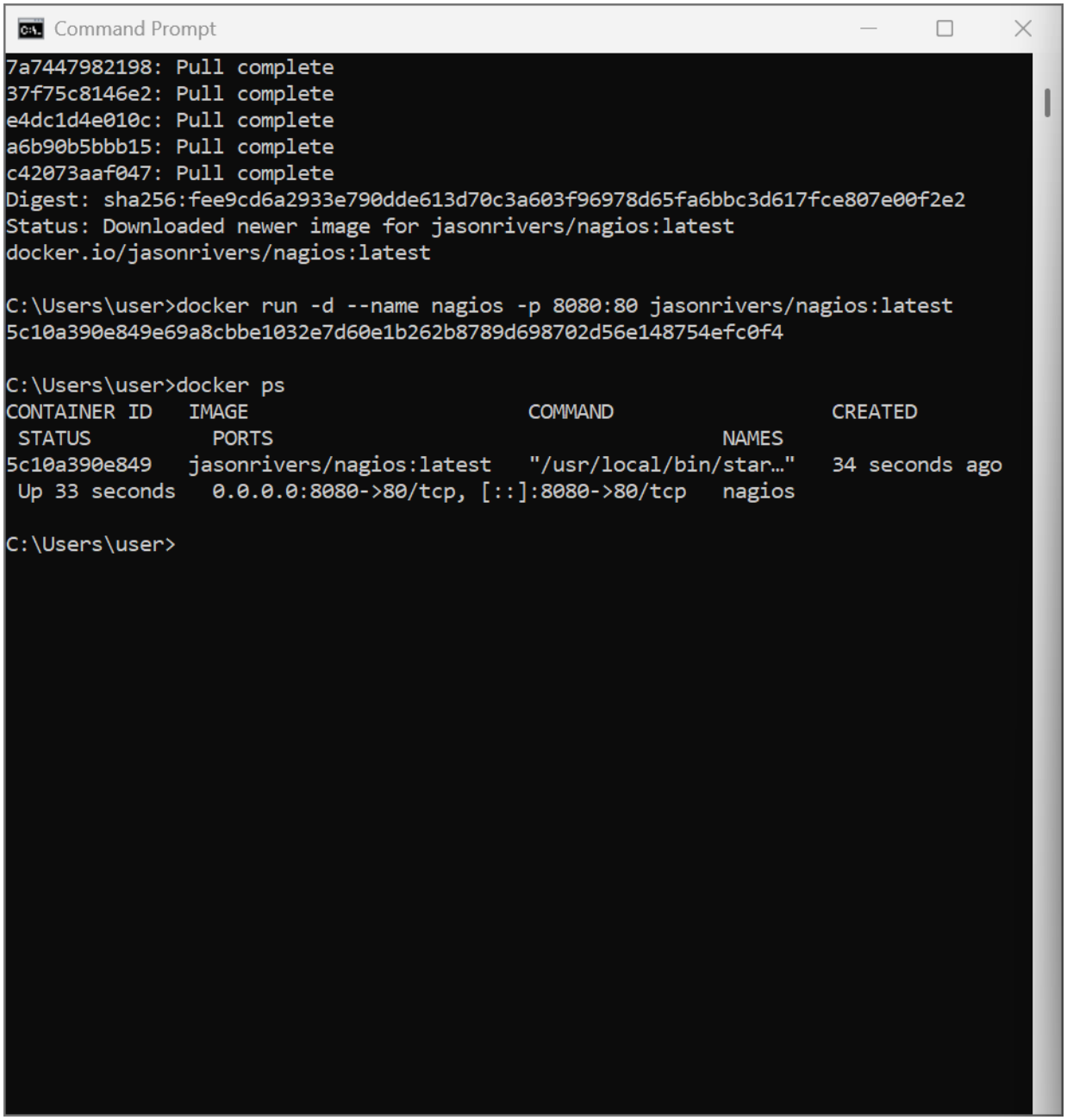


Fig. 4: Nagios container running and port 8080 mapped

3. Access Nagios Web Interface

The Nagios web interface was accessed from the Windows host using the following address:

```
http://localhost:8080
```

The Nagios Core interface opened successfully. The dashboard showed Nagios Core version 4.5.12 and indicated that the Nagios process was running.

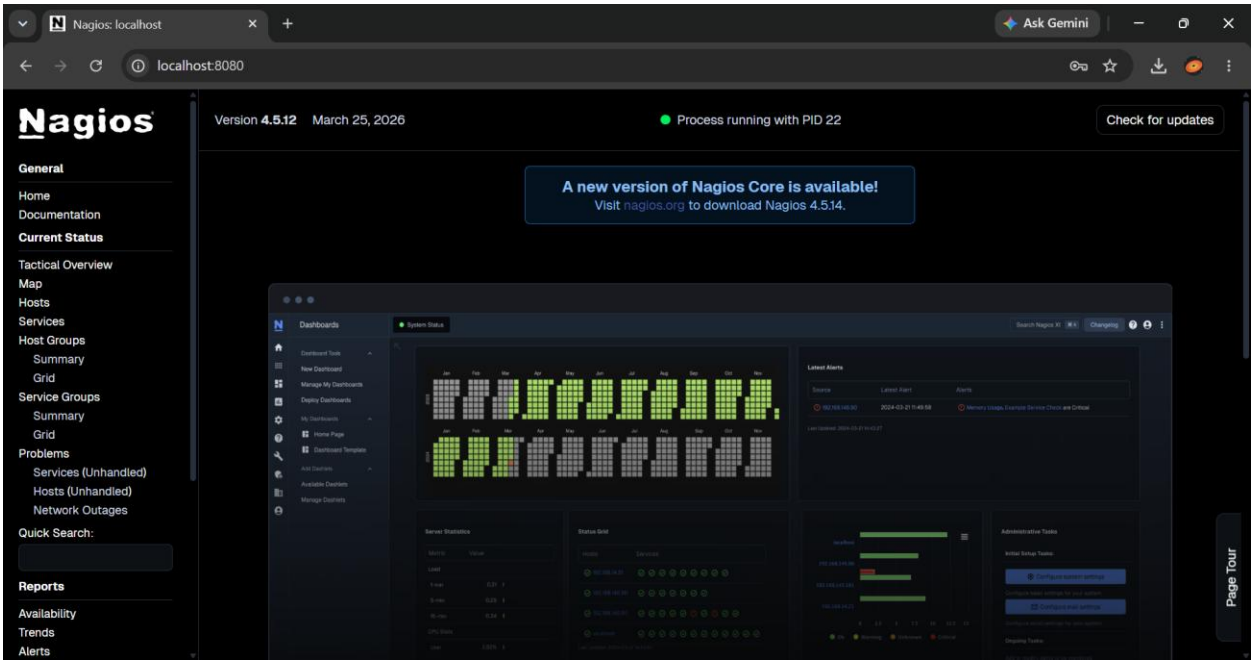


Fig. 5: Nagios Core web interface running successfully

4. Windows Host Monitoring using NSClient++

The experiment handout describes NSClient++ as the Windows counterpart of NRPE for monitoring Windows hosts. The procedure includes installing NSClient++, configuring the allowed hosts list, restarting the NSClient++ service, verifying NRPE connectivity, and adding a Windows host and PING service to Nagios.

The screenshots captured for this submission document the WSL2, Docker, Nagios image, container, and Nagios web-interface stages. No separate NSClient++ execution screenshot was captured in the available experiment evidence.

5. Host Configuration Format

The experiment handout specifies the following Nagios host and PING service configuration:

```
define host {
    use             windows-server
    host_name       Windows_Host_001
    alias           Windows Host 001
    address         192.168.1.50
    register        1
}

define service {
    host_name       Windows_Host_001
    service_description PING
    check_command    check_ping!100.0,20%!500.0,60%
    max_check_attempts 2
    check_interval    2
    retry_interval    2
    check_period      24x7
    check_freshness   1
```

```
    contact_groups    admins
    notification_interval 2
    notification_period 24x7
    notifications_enabled 1
    register         1
}
```

6. Verification

The Docker container was verified using docker ps and the Nagios web interface was successfully opened at localhost:8080. The Nagios process was shown as running in the web interface.

```
docker ps
http://localhost:8080
```

Result

Nagios Core was successfully deployed in a Docker container on Windows using the WSL2 backend. The container was verified as running, and the Nagios web interface was successfully accessed through localhost:8080.

Conclusion

Thus, the monitoring experiment was successfully implemented using Nagios with Docker and WSL2. The experiment demonstrated the setup of a monitoring environment, execution of Nagios in a Docker container, verification of the running container, and access to the Nagios monitoring interface. Nagios can be used to monitor hosts, services, and infrastructure availability.

Findings

The WSL2 environment and Docker Engine were successfully configured. The Nagios Docker image was downloaded, the Nagios container remained in the running state, and the Nagios Core web interface was accessible from the Windows browser.